Part 01 Questions

Question.1: What is the purpose of the finally block?

Answer: The finally block is used to execute code that must run regardless of whether an exception was thrown or caught. It is typically used for cleanup operations, such as closing database connections, file streams, or releasing managed/unmanaged resources.

Question.2: How does int.TryParse() improve program robustness compared to int.Parse()?

Answer: int.TryParse() prevents program crashes by returning a boolean indicating success or failure instead of throwing a FormatException or OverflowException when the input is invalid.

Question.3: What exception occurs when trying to access Value on a null Nullable<T>?

Answer: An InvalidOperationException is thrown.

Question.4: Why is it necessary to check array bounds before accessing elements?

Answer: To prevent an IndexOutOfRangeException at runtime, which occurs when attempting to access an index less than zero or greater than/equal to the array's total length.

Question.5: How is the GetLength(dimension) method used in multi-dimensional arrays?

Answer: GetLength(dimension) returns the number of elements in a specific dimension of a multi-dimensional array (e.g., GetLength(0) for rows and GetLength(1) for columns), allowing safe iteration.

Question.6: How does the memory allocation differ between jagged arrays and rectangular arrays?

Answer: A rectangular array (e.g., int[,]) is allocated as a single contiguous block of memory. A jagged array (e.g., int[][]) is an array of arrays, where each sub-array is allocated independently on the heap and can have different lengths.

Question.7: What is the purpose of nullable reference types in C#?

Answer: They help prevent NullReferenceException bugs at compile-time by forcing developers to explicitly declare whether a reference variable can hold a null value or not (string? vs string).

Question.8: What is the performance impact of boxing and unboxing in C#?

Answer: Boxing requires allocating memory on the Heap and copying the value type. Unboxing requires type-checking and copying the value back to the Stack. Frequent boxing/unboxing causes CPU overhead and increases Garbage Collection pressure.

Question.9: Why must out parameters be initialized inside the method?

Answer: The C# compiler enforces that an out parameter must be assigned a value inside the method before returning, ensuring that the caller receives a guaranteed initialized variable.

Question.10: Why must optional parameters always appear at the end of a method's parameter list?

Answer: Optional parameters must be placed at the end so the compiler can unambiguously match positional arguments passed during a method call.

Question.11: How does the null propagation operator (?.) prevent NullReferenceException?

Answer: It short-circuits evaluation: if the operand on the left side is null, it immediately returns null instead of attempting to access member properties/methods.

Question.12: When is a switch expression preferred over a traditional if statement?

Answer: Switch expressions are preferred when mapping a value directly to a result, offering cleaner, pattern-matching syntax and concise, functional-style code.

Question.13: What are the limitations of the params keyword in method definitions?

Answer: Only one params parameter is allowed per method, and it must be the last parameter in the method signature. It must also be a single-dimensional array type